

How We Diagnosed and Fixed the 2023 Voyager 1 Anomaly from 15 Billion Miles Away

David Cummings
Jet Propulsion Laboratory
California Institute of Technology
25-March-2025.



Jet Propulsion Laboratory
California Institute of Technology

© 2025 California Institute of Technology. Government sponsorship acknowledged.

"This document has been reviewed and determined not to contain export controlled CUI."

Presentation Overview

- The anomaly
- How we diagnosed it
- How we fixed it

All of this was possible only through the collective effort of the entire team working together and supporting one another. This was truly a team effort.

The Anomaly

- On Nov. 14, 2023, VGR-1 stopped sending meaningful data
- We were receiving a signal, but it was a string of all 1's or all 0's
- By the time I joined the team (about a week later), the team had already concluded that the FDS (Flight Data Subsystem) was the most likely cause
- The FDS is one of 3 subsystems with processors and associated flight software. Also:
 - CCS (Computer Command Subsystem)
 - AACS (Attitude and Articulation Control Subsystem)

How We Diagnosed It

FDS Block Diagram

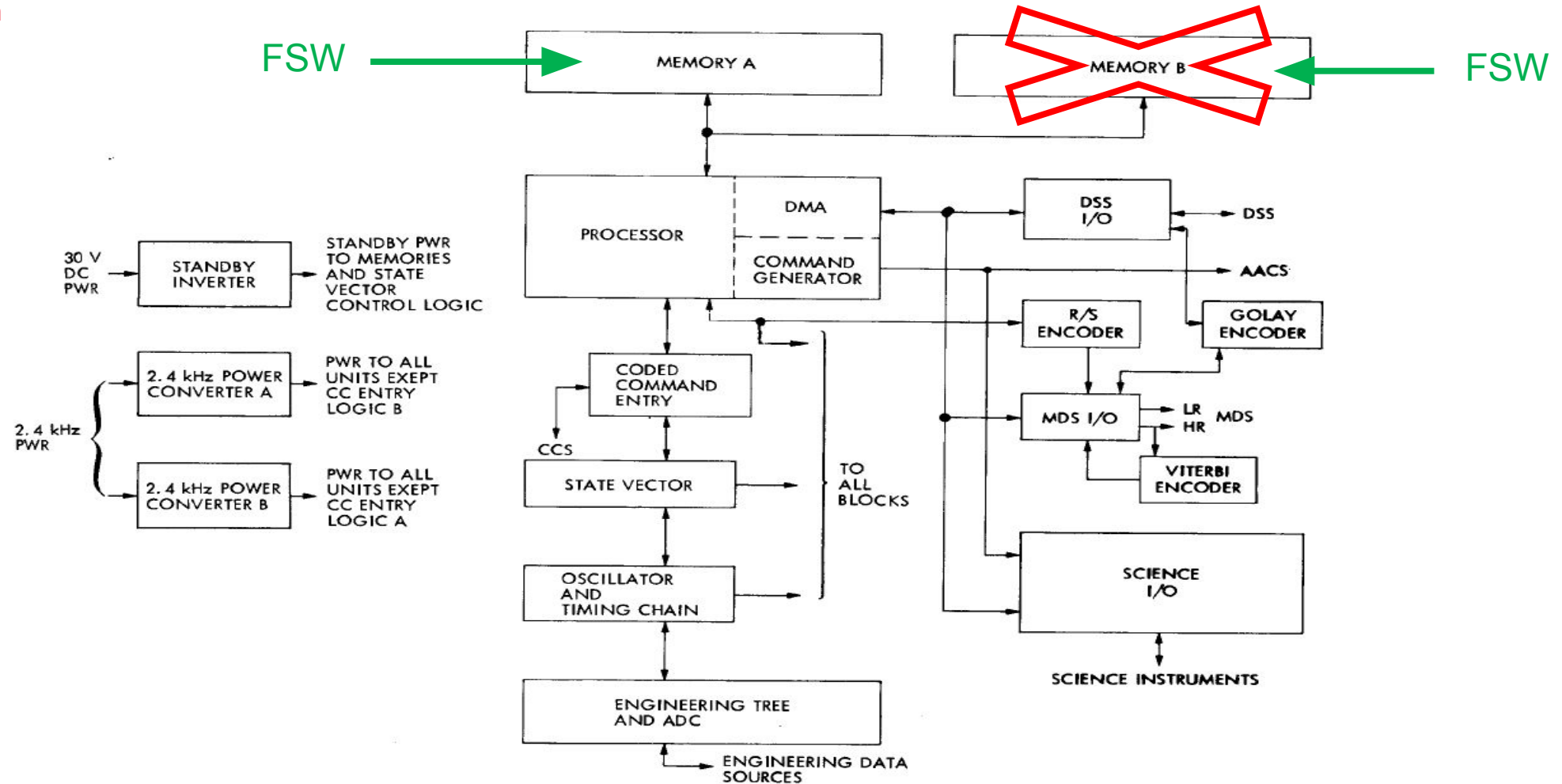


Figure 1. FDS Hardware Functional Block Diagram

How We Diagnosed It

Commands that were tried

- A number of commands were sent during the first few months of the anomaly
- Resulted in observable behavior that provided some clues, but still no telemetry
 - Commands that resulted in expected carrier frequency shifts
 - Commands to change the subcarrier frequency
 - Swapping between various redundant units (both within and outside FDS)
- Voyager 1 is more than 15 billion miles from Earth
 - At the time of the anomaly investigation, one-way light time was about 22.5 hours
 - Send command and then wait 45 hours to see the results

How We Diagnosed It

Focus on FDS flight software

- By early January, we were looking more closely at the FDS flight software
- It was seeming increasingly likely that either
 - The FDS FSW had failed in some way due to a processor error, or
 - The FDS processor was sound, but the FSW had failed in some way due to a memory error
- Swapping to the backup FDS processor was possible, but
 - It was deemed risky
 - We felt a memory error was more likely

How We Diagnosed It

FDS FSW Structure

- Every 2.5 ms an interrupt fires that restarts FSW execution at location zero
- Therefore, FSW is broken up into 24 sequential chunks of execution called “P periods”, 2.5 ms each
- There is a jump table (“P Table”) with pointers to the code for each P period (pointers to the 24 “P routines”)
- There is a counter (“P counter”) that keeps track of the next P routine to execute
- One complete cycle through the 24 P routines takes $24 \times 2.5 \text{ ms} = 60 \text{ ms}$ (a “line”)
- 800 lines is called a “frame”, and it requires $800 \times 60 \text{ ms} = 48 \text{ seconds}$

How We Diagnosed It

P Tables and data mode

- There are 10 different “data modes” that the spacecraft can execute
 - Each data mode can be thought of as a different program to execute
- Which data mode the spacecraft is currently executing is determined by a ground command
- In order to support these 10 different data modes, there are 4 P Tables
 - First P Table supports GS-4B, GS-8, PB-5, PB-14, PB-15
 - Second P Table supports just EH-12
 - Third P Table supports just EL-40
 - Fourth P Table supports UV-5A, PB-16, CR-5T

How We Diagnosed It

FDS FSW Structure

0000	9FCF		ISZ	PCTR	/INCREMENT P-COUNTER FOR THIS P-PERIOD	V5001160
0001	801F		ABS	31	/FORCE IT TO BE BETWEEN 0 AND 31	V5001180
0002	93E0		AND	PCTR,R1	/ INCLUSIVE BY KEEPING ONLY 5LSB	V5001190
0003	9C52		SKP	CMODE	/IS LOWER MEMORY P-TABLE TO BE USED	V5001200
0004	002B		JMP	GOUP	/ NO	V5001210
					/ YES	V5001220
0005	8009		ABS	PTAB	/GENERATE ADDRESS WITHIN PTAB TABLE	ECR100073
0006	900F		ADD	R1,PCTR	/V5001240	
0007	0FC0		JMI	R1	/ AND JUMP TO IT	V5001250
0008	11AF	PROGID	CON	\$X11AF	/PROGRAM ID	V5001260
0009	0028	PTAB	JMP	PILL	/ILLEGAL	V5001310
000A	002D		JMP	P1	/V5001320	
000B	0185		JMP	P2	/V5001330	
...						
0020	0486		JMP	P23	/V5001540	
0021	048F		JMP	P24	/V5001550	
0022	0028		JMP	PILL	/ILLEGAL	V5001560
...						
0028	91FF	PILL	LXR	PCTR,PCTR	/RESET P-COUNTER TO 0	V5001660
0029	9F86		ISZ	PILLCT	/INCREMENT ILLEGAL P-PERIOD COUNTER	V5001680
002A	3FFD	PX	WAT	4095	/WAIT FOR NEXT CLOCK-400 INTERRUPT	V5001760
002B	F087	GOUP	OUT	7,SETJU	/SET JUMP UP	V5001800
002C	0000		JMP	UPINT	/JUMP TO UPPER MEMORY INTERRUPT	V5001810
...						
1000	F097	UPINT	OUT	7,SETAU	/SET ADDRESS UP	V5060390

How We Diagnosed It

FDS FSW Structure – MODE, CMODE, and additional P Tables

- Three additional P Tables, all in upper memory
- Two variables in memory determine which data mode, and hence which P Table, is executed
 - MODE, in write-protected memory
 - CMODE, in read/write memory

0BE5 81C0	MODE	CON	\$B1000000111000000	/LOAD-STATE IS EL, NO PPS SAFING, ENG	V5052620
				/ MODE 7 (MODIFIED CRUISE TABLE AND	V5052630
				/ AACS MANEUVER (50) TABLE), AACS TEL	V5052640
...					
0FD2 FFE0	CMODE	CON	\$XFFE0	/CURRENT MODE (IN HEX), WHERE	V5058260
0001	GS5	EQU	1	/ 1 = GS-5	V5058270
0003	GS4B	EQU	3	/ 3 = GS-4B PWS REPLACEMENT	V5058280
0007	GS8	EQU	7	/ 7 = GS-8	V5058290
000B	PB5	EQU	11	/ B = PB-5	V5058300
000C	PB14	EQU	12	/ C = PB-14	V5058310
000F	PB15	EQU	15	/ F = PB-15	V5058320
				/ FFE0 = EL	V5058330
				/ FFE1 = EH	V5058340
				/ FFF2 = UV-5A (UV-5)	V5058350
				/ FFF3 = PB-16	V5058360
				/ FFF5 = CR-5T (AKA CR-5A)	V5058370

How We Diagnosed It

FDS FSW Structure – MODE, CMODE, and additional P Tables

- MODE, in write-protected memory, is only set via ground command
- CMODE, in read/write memory, is derived based on the value in MODE:

053C	1C08	SETMO	SRB	LMRA	/SAVE RETURN ADDRESS	V5031230
053D	9B20		SLC	800	/IS LINE COUNTER AT 800	V5031240
053E	0543		JMP	SETMOA	/ YES	V5031250
					/ NO	V5031260
053F	8000		ABS	0	/RESET SKIPPR HERE BECAUSE IT IS A	V5031270
0540	4F83		MLD	SKIPPR	/ CONVENIENT TIME TO DO IT	V5031280
0541	305D		WAT	95	/EQUALIZE TIME WITH L800 PATH	V5031290
0542	2C08		EXC	LMRA	/RETURN	V5031300
					/LINE COUNTER IS AT 800	V5031310
0543	5BE5	SETMOA	MRD	MODE	/GET MODE WORD FOR UPCOMING 48 SECONDS	V5031320
...						

- Only reference in source code to “MODE” variable is the MRD instruction at location 0543
 - Thus, the ground command writes directly to the MODE variable in write-protected memory
 - Also, SETMO is called by P24 (the very last P routine) in each of the four P Tables
- CMODE is then used by FSW to select P Table and data mode within that P Table

How We Diagnosed It

Memory corruption hypothesis

- Assuming it was some sort of memory problem, we hypothesized:
 - A single bit flip in one of the instructions in one of the P routines
 - This caused that P routine to somehow get “stuck”

How We Diagnosed It

First of two parallel activities to pursue FDS memory corruption hypothesis

- Create a small “Hello World” program to transmit simple telemetry to show that:
 - FDS processor was operational
 - At least some portion of FDS memory was operational
- Then morph it into a program to downlink an FDS full memory readout (MRO)
- Saw references in old documents to “MIN CMROT” (Minimum Command Routine)
 - Included minimal telemetry, which was of particular interest
 - Quickest path to “Hello World”
- No source code could be found
- Roger Ludwig found this:

JET PROPULSION LABORATORY INTEROFFICE MEMORANDUM

VOYAGER SCT-98-008

27 February 1998

TO: Distribution

FROM: R.Ellis

SUBJECT: Procedure for Loading VIM7 Flight Software Into the Flight Data Subsystem (Revised)

How We Diagnosed It

First of two parallel activities to pursue FDS memory corruption hypothesis

Table 4 Minimum CMROT Macro Call

```
FDSABW P,(0E72,5F7A,,9FC0,,E401,,9551,,1F7A,,8E83,,9001,,0FC0,,;  
8000,,4F7A,,8001,,4FD4,,4FD3,,4F7D,,8F6F,,4F74,,;  
3FFD,,0E7A,,0EA3,,0E82,,0E82,,0EAA,,0E82,,0E82,,;  
...  
4F77,,3003,,0F70,,7310,,0F70,,0F6A,,1F74,,0E82,,;  
301F,,0E82,,0F6F,,0391,,5ED3,,0000,,8400,,BA00,,; 001A,,0000,,0000)
```

- We wondered if this was the MIN CMROT object code (267 words)
- By mid January we had disassembled and reverse-engineered it
 - Confirmed that it was indeed object code for MIN CMROT
 - Identified a few instructions that appeared to be sending telemetry
- BUT, to uplink this, we needed to patch 267 words of FDS memory
 - Without functional FSW, that meant using a “hardware memory load” (“hardware poke”)
 - This had never been used after launch because it was deemed too risky

How We Diagnosed It

Second of two parallel activities to pursue FDS memory corruption hypothesis

- Command the spacecraft to execute each of the 10 data modes
 - Force the FSW to break out of its hypothesized “stuck” state and do something, anything
 - “Anything” because some of the data modes transmit data at rates too high to be received
- Unfortunately, there is a lot of shared code
 - Subroutines called by all P Tables
 - One P routine sometimes jumps into a different P routine
 - Sometimes across P Tables
- Therefore, the hypothesized bit flip could cause more than one P Table to get “stuck”
 - Still, we thought it was worth a try

How We Diagnosed It

Commanding the different data modes

- To command the spacecraft to attempt to execute each of the 10 data modes, we used this:

7.4 SC06BB FDS MODE COMMAND

DESTINATION

FDS

EFFECT ON SPACECRAFT

This command updates FDS memory location MODE in primary memory which controls the basic data mode of the FDS.

- As we prepared the 10 SC06BB commands for uplink, we realized would not work because:
 - Analysis and evidence suggested that P24 was never executing
 - If P24 were not executing, then SETMO would never be called and CMODE would never be updated
 - Also, learned that the SC06BB command relied on the FSW in order to write to the MODE variable
 - But FSW was not functional

How We Diagnosed It

Commanding the different data modes

- Solution:
 - Use hardware pokes to change MODE
 - Use hardware pokes to force CMODE to the correct value

How We Diagnosed It

The big day

- Uplinked the 10 pairs of hardware poke commands on Friday, Feb. 23
 - Each data mode was given about 10 minutes to run before we commanded the next one
- Waited with bated breath for the results on Sunday, Feb 25
- VERY DISAPPOINTING:

From: Chin, Greg E (US 393E)

Sent: Sunday, February 25, 2024 11:25 AM

To:

Subject: V31 - Status - Post H/W Poke

Hi –

Jack Lippencott, VGR NOPE, supported today's activity – and he will be generating a post-track summary and providing additional details from the DSN.

However, from my observation (log):

<List of commands sent and negative results for each one>

So, at this time, no joy.

-- Greg Chin

ACE, Science Data Processing, ISSO, or Ad Hoc Assignment du Jour, Voyager Interstellar Mission

GDS Engineering/Documentation, Voyager Interstellar Mission

Coordinator, Multi-Mission Activity Planning Teleconference (MMAPT)

How We Diagnosed It

A ray of hope

From: Mellstrom, Jeffrey A (US 7000)
Sent: Monday, February 26, 2024 9:20 AM
To:
Subject: 2/26/2024 Agenda, Voyager 1 Tiger Team

Agenda:

Review OLR results from Sunday: Oscar Yang

Looks like something is in the OLR results

Next steps discussion: Options, risks, decision

If time, HW discussion options:

Thang's HW insights regarding memory load

Jim's insights into failure modes

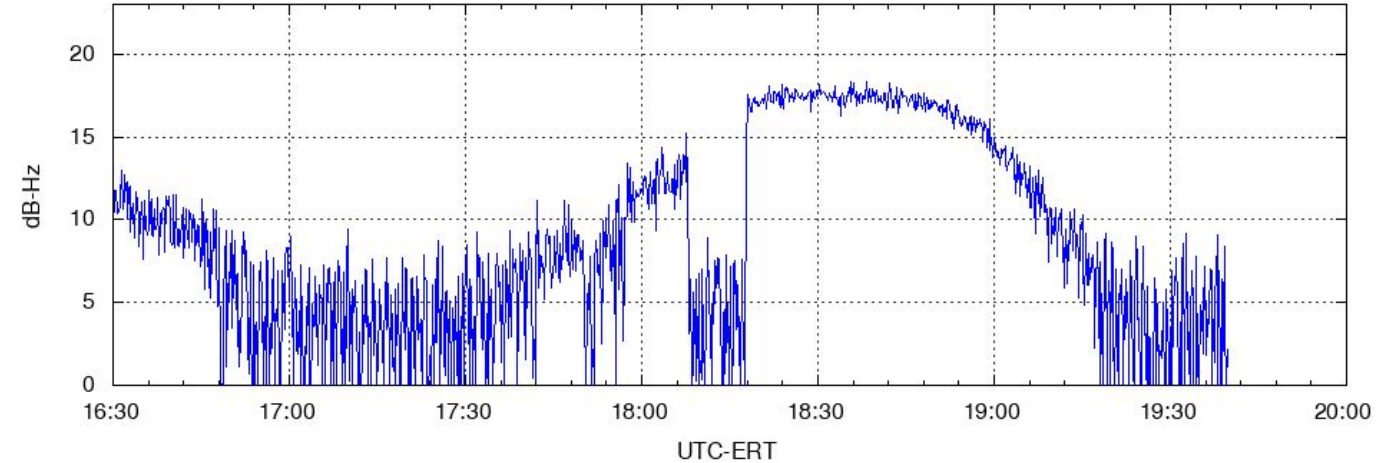
How We Diagnosed It

A ray of hope

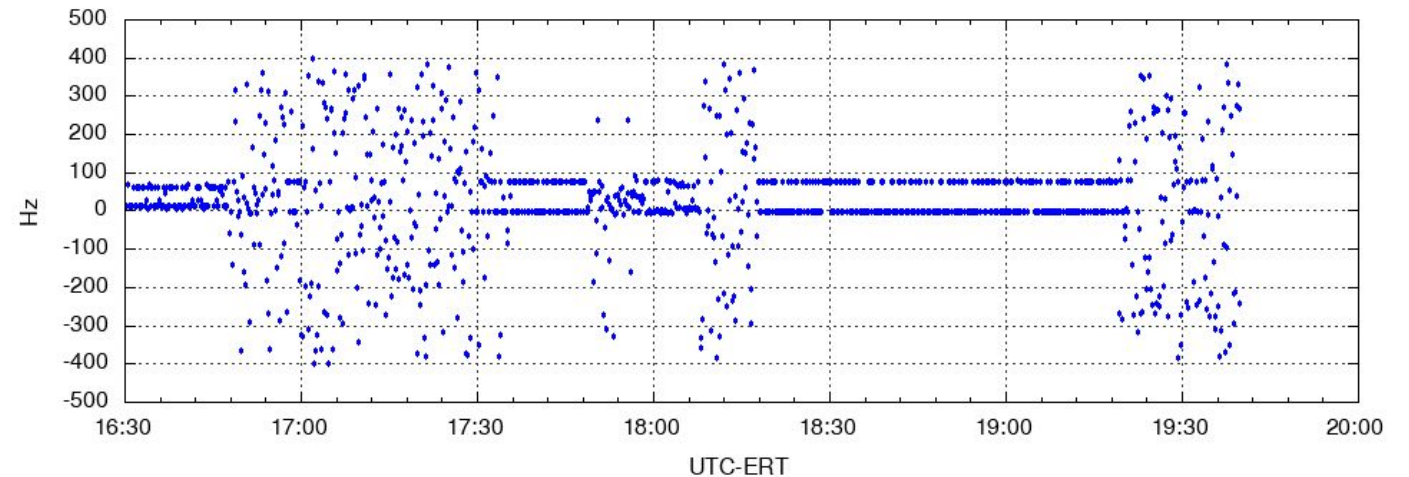
SFOS Times:

- 16:47 GS-08 2.8 KBPS MI=72 DEG
- 16:57 EH-12 1.2 KBPS MI=70 DEG
- 17:07 GS-4B 2.8 KBPS MI=72 DEG
- 17:17 CR-5T 160 BPS MI=60 DEG
- 17:27 PB-16 4.8 KBPS MI=72 DEG
- 17:37 UV-5A 640 BPS MI=65 DEG
- 17:47 PB-15 2.8 KBPS MI=72 DEG
- 17:57 PB-14 1.4 KBPS MI=70 DEG
- 18:07 PB-05 7.2 KBPS MI=72 DEG
- 18:17 EL-40 40 BPS MI=58 DEG
- 18:34 SUBC HIGH FRAME TRAP

2024/056 VGR1 Subcarrier 10-sec integration DSS-43 X LCP Signal-to-Noise Ratio



2024/056 VGR1 Subcarrier 10-sec integration DSS-43 X LCP Residual Frequency



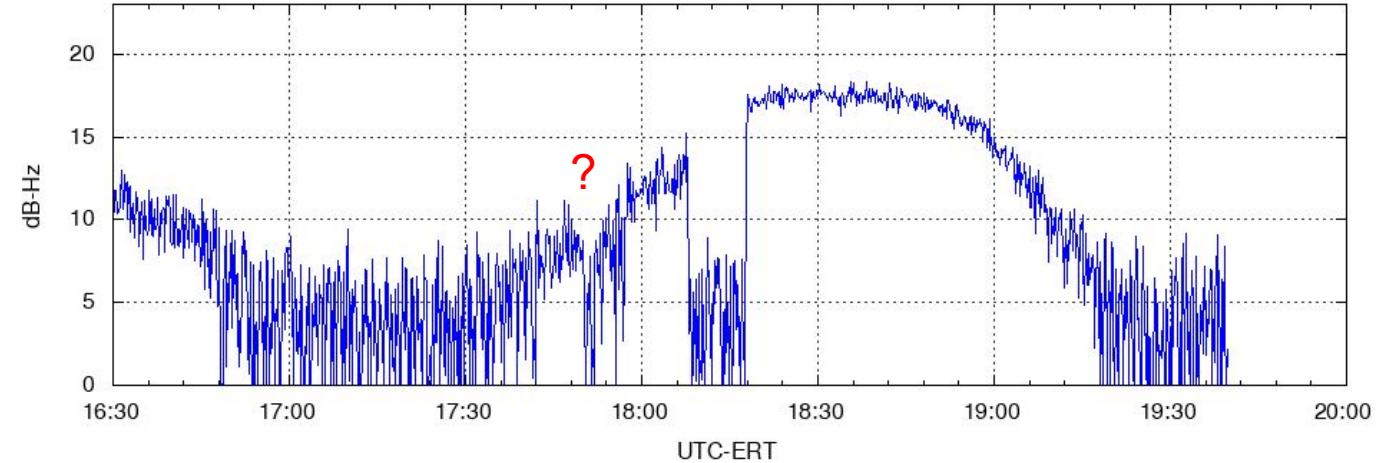
How We Diagnosed It

A ray of hope

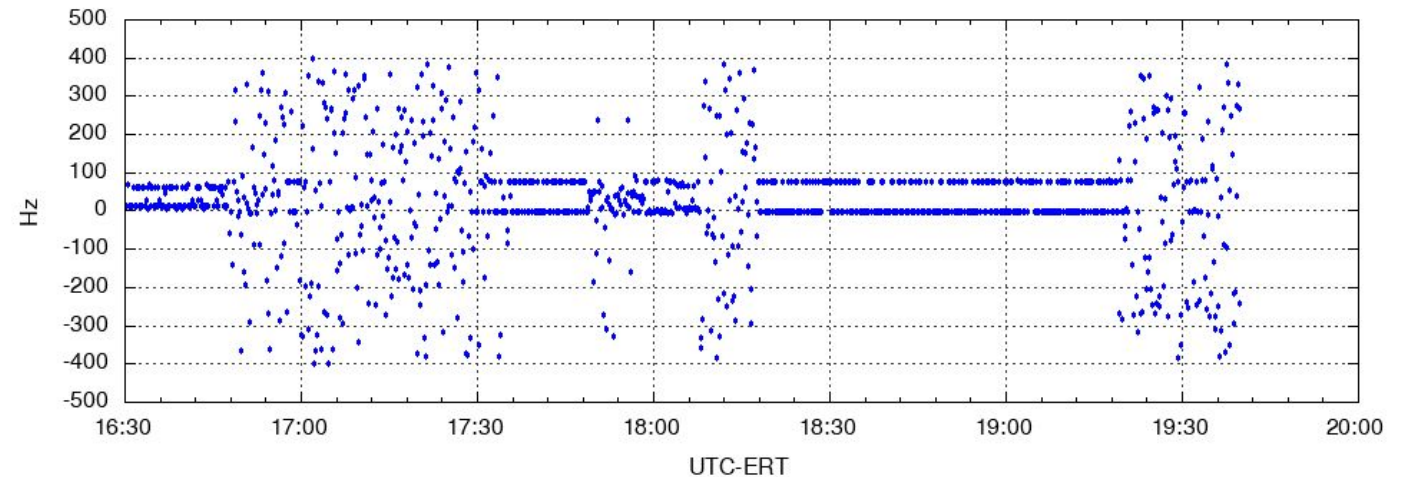
SFOS Times:

- 16:47 GS-08 2.8 KBPS MI=72 DEG
- 16:57 EH-12 1.2 KBPS MI=70 DEG
- 17:07 GS-4B 2.8 KBPS MI=72 DEG
- 17:17 CR-5T 160 BPS MI=60 DEG
- 17:27 PB-16 4.8 KBPS MI=72 DEG
- 17:37 UV-5A 640 BPS MI=65 DEG
- 17:47 PB-15 2.8 KBPS MI=72 DEG
- 17:57 PB-14 1.4 KBPS MI=70 DEG
- 18:07 PB-05 7.2 KBPS MI=72 DEG
- 18:17 EL-40 40 BPS MI=58 DEG
- 18:34 SUBC HIGH FRAME TRAP

2024/056 VGR1 Subcarrier 10-sec integration DSS-43 X LCP Signal-to-Noise Ratio



2024/056 VGR1 Subcarrier 10-sec integration DSS-43 X LCP Residual Frequency



How We Diagnosed It

Breakthrough!

- Subsequent analysis of the OLR data further confirmed the response to the ground commands
 - Evidence of signal modulation that lined up with the timing of the commands:

From: O'Dea, Andrew (US 333F)

Sent: Tuesday, February 27, 2024 4:55 PM

To:

Subject: Re: VGR1 2024/056 DSS-43 OLR Analysis

Hi Jeff,

These charts are a little rough, but I think they show something close to BPSK modulation of random data for PB-15, especially compared to what we see for PB-14 during the next ten-minute period. These plots match what Danny's subcarrier residual frequency charts show, but as spectrum snapshots.

- Andrew

- Breakthrough! Compelling evidence (confirmed by 3/1/2024 commanding) that:
 - The FDS processor was working
 - At least some portion of Memory A was sound
 - The anomaly was due to a memory corruption
 - The hardware poke worked correctly

How We Diagnosed It

The memory corruption

- Friday, March 1
 - Commanded 3 data modes, and let each run for an hour before going to the next one
- Thursday, March 7
 - Andrew O'Dea decoded data from the subsequent OLR
 - Looked like portions of a memory dump
- Friday, March 8
 - Andrew decoded data from another downlink pass
- Saturday, March 9
 - Analysis of his decoded data LOOKED LIKE A FULL MEMORY DUMP
 - Further analysis revealed 256 contiguous words peppered with corruptions (3% of FDS memory)
 - Addresses 1400 – 14FF
 - These locations contained subroutines (R/O instructions) shared by ALL of the P Tables
- Sunday, March 10
 - Problem traced to a single RAM chip responsible for the 5th bit in each location
 - All bits stuck at 1

How We Diagnosed It

The memory corruption

13DE	1C00	GETEID	SRB	UMRA	/SAVE RETURN ADDRESS	V5079310
...						
1400	7829 7839		PWD	RMFC,R2	/CALCULATE ADDRESS AND GET IDS FOR 400	V5079770
...						
1461	1C09 1C19	MOD15	SRB	LMRAN	/SAVE RETURN ADDRESS	V5081080
...						
146D	F09F	COPY	OUT	7,SETAD	/SET ADDRESS DOWN	V5081380
...						
1494	F09F	SAFECA	OUT	7,SETAD	/SET ADDRESS DOWN	V5081990
...						
14B0	5F84 5F94	PLSCL	MRD	AUTO CF	/PLS AUTO CAL FLAG	V5082500
...						
14CA	1C08 1C18	PLSCHK	SRB	LMRA	/SAVE RETURN	V5082830
...						
14F0	37AF 37BF	PLSTBL	CON	\$X37AF		/V5083250
14F1	ADFA		CON	\$XADFA		/V5083260
14F2	FA7F		CON	\$XFA7F		/V5083270
14F3	AFB0	PLTZ	CON	\$XAFB0	/0' NOT USED	V5083280
...						
14F4	F09F	ULINK	OUT	7,SETAD	/SET ADDRESS DOWN	V5083400
...						
14F8	03DE	XGETEI	JMS	GETEID	/GET ENG ID SUBROUTINE	V5083470
14F9	050A 051A		JMP	UVECTR	/RETURN TO CALLER IN LOWER MEMORY	V5083480
14FA	0231	XCMROT	JMS	CMROT	/CODED COMMAND PROCESSING	V5083490
14FB	050A 051A		JMP	UVECTR	/RETURN TO CALLER IN LOWER MEMORY	V5083500
14FC	02B6	XCHSUM	JMS	CHSUM	/CHECKSUM	V5083510
14FD	050A 051A		JMP	UVECTR	/RETURN TO CALLER IN LOWER MEMORY	V5083520
14FE	02EF 02FF	XLTS	JMS	LTS	/LTS SUBROUTINE	V5083530
14FF	050A 051A		JMP	UVECTR	/RETURN TO CALLER IN LOWER MEMORY	V5083540

GETEID extends to address 1460

Corrupted memory extends from addresses 1400 – 14FF

The 5th bit of each word from 1400 – 14FF is stuck at 1

How We Diagnosed It

MIN CMROT and MINMRO

- Wednesday, March 6
 - Morphed MIN CMROT (“Hello World”) into MIN MRO and prepared MIN MRO patch
- Friday, March 8
 - Uplinked and ran MIN CMROT
 - It worked! We could see the expected telemetry.
- Friday, March 15
 - Uplinked and ran MIN MRO
 - It worked!
 - For any future anomalies where FDS memory is suspect, MIN MRO can be used

The Fix

"This document has been reviewed and determined not to contain export controlled CUI."

Fix overview

- Find a 256-word section of memory that is available
- Relocate the 256 instructions from the corrupted addresses (0x1400 - 0x14FF) to the available memory
- Identify all places in the rest of the code that reference the code being relocated
 - Calls, jumps, or data accesses
- Fix up those references to refer to the new (relocated) addresses
- Use upper memory (upper 4K words) for relocation
 - Reduces risk
 - Calls/jumps/accesses between upper memory and lower memory add complexity
 - Also affects execution timing

Challenges

- JPL designed processor with custom instruction set
 - A document that may or may not be accurate
- No source code files
 - Source code listing in Microsoft Word with errors
 - May or may not match the version on the spacecraft
- No assembler
- No simulator
- No testbed
- Telemetry must be restored as soon as possible!!!
 - No visibility into an aging and delicate spacecraft
- AND: We were unable to find 256 contiguous words of memory that were unused!

JET PROPULSION LABORATORY	INTEROFFICE MEMO MJS: 2.64A
	7 October 1974
TO:	MJS FDS Distribution
FROM:	J. Wooddell
SUBJECT:	MJS FDS Processor Architecture and Instruction Set
SUMMARY:	The architecture and instruction set of the MJS FDS Processor are described in the following pages.

Addressing those challenges

- Do it in 2 phases
 - Phase 1, Engineering Telemetry Mode: EL-40
 - Phase 2, Science Telemetry Modes: GS-4B, PB-14, CR-5T
- Sacrifice EH-12 code - High-Rate (1200 bps) Engineering-Only data mode
 - Gave us the required 256 available words in upper memory
 - BUT not contiguous, due to sharing of code between EH-12 and other data modes
- Modify binary instructions
 - Too time-consuming to develop and certify assembler
- Test by inspection
 - Too time-consuming to develop and certify simulator
 - Enlisted help from Matt Wette and Armen Arslanian
 - Over time, created an extensive checklist, and iterated over it repeatedly
 - Accepted that we will never achieve 100% confidence

Phase 1: Engineering Telemetry Patch

- Patch 1: Relocate corrupted code referenced by EL-40
- Patch 2: Fix up all EL-40 references to corrupted code

Phase 1: Engineering Telemetry Patch

Patch 1: Relocate corrupted code reference by EL-40

- Find contiguous subsets of EH-12 code that are available as relocation destinations
 - Must not be used (shared) by any other data mode
 - Required tracing backwards through all call trees
 - Reliance on inspection to accurately identify shared code was error-prone
 - We knew there were errors in the source code listing
 - Therefore, could not rely solely on labels to identify sharing

11BD	00D6	Z	JMP	ELP21
11BE	00D4	Z	JMP	ELP22
11BF	01E8	Z	JMP	ELP23
11C0	01EB	Z	JMP	ELP23

- Thus, also used object code inspection to identify sharing, but that was not foolproof either
 - If we overlooked some sharing, then relocation to that area would break other data modes
- Identify corrupted subroutines that are referenced by EL-40
 - Required similar back-tracing through code
 - If we overlooked some EL-40 references, EL-40 could fail unpredictably

Phase 1: Engineering Telemetry Patch

Patch 1: Relocate corrupted code reference by EL-40

- Examples of potential pitfalls:
 - Beware of JMP .+n, JMP label+n, JMP .-n, JMP label-n
 - Beware of skip instructions (10 of them in this instruction set)

```
103C 9F8B      EHP1A  ISZ  S96SI      /INCREMENT 96-SECOND INDICATOR
103D 0054  Z           JMP  EHP1Z
103E 9094      EHP1L1 LXR  R20LC,R20LC  /ZERO 20-LINE COUNTER
```

- Must include redefinition of labels (EQU) and constants (CON)

Phase 1: Engineering Telemetry Patch

Patch 1: Relocate corrupted code reference by EL-40

- For relocation, needed to break up subroutines to fit into the contiguous areas of EH-12
 - Add jumps, which adds clock cycles
 - Must not break up code after a skip instruction

1461	1C09	MOD15	SRB	LMRAN	/SAVE RETURN ADDRESS
...					
1469	9C40		SKP	R1	/IF DIFF IS POSITIVE USE IT
146A	9D83		ISP	SKIPPR	/ NOT POSITIVE, SO DO A 4-CYCLE SKIP
146B	7820		PWD	R1,R2	/ POSITIVE SO USE IT

- Must not break up code at an instruction that relies on register B (SRB, PWD, AML)

14B0	5F84	PLSCL	MRD	AUTO CF	/PLS AUTO CAL FLAG
...					
14BF	5BED		MRD	PLSCWB	/RESTORE NORMAL CONFIGURATION
14C0	1C31		SRB	NEWPLS	
14C1	04C8	Z	JMP	GSNOCA	/CONTINUE

- Modify relocated code as needed (mainly references to labels at new locations)

Phase 1: Engineering Telemetry Patch

Patch 2: Fix up all EL-40 references to corrupted code

- Modify each EL-40 reference that was found so it refers to new address of relocated subroutine or data
- Special case: Calls to COPY subroutine, which was in corrupted memory
 - Copies memory contents from memory A to memory B
 - Unneeded, because Voyager 1's FDS memory B deemed unusable since 1981
 - Therefore, NO-OP the COPY subroutine instead of relocating it
 - Replace "JMP COPY" with "WAT 344" to preserve timing
 - BUT, calls to COPY have a side effect
 - COPY subroutine sets the processor to lower address mode
 - Leaves it in that mode when it exits
 - Analyzed all callers of COPY, and determined that none relied on this side effect
 - Therefore, safe to just NO-OP each call

Phase 1: Engineering Telemetry Patch

- Patch 1 and Patch 2 each consisted of a set of <address, value> pairs to be uplinked
- Our checklist covered all of the potential pitfalls discussed above for developing Patch 1 and Patch 2 (and more), plus additional verifications for the <address, value> pairs such as:
 - All code required to be moved is moved
 - Relocation destinations must be free (used only by EH-12)
 - Relocated code must fit properly (whether it has been broken into pieces or not)
 - Addresses that one section of code is relocated to must not conflict with addresses that other sections of code are relocated to
 - Relocated addresses for each subroutine or piece thereof are consecutive
 - Repeat all of the above (and more) until the 2 patches were uplinked!

Phase 1: Engineering Telemetry Patch

April 20, 2024

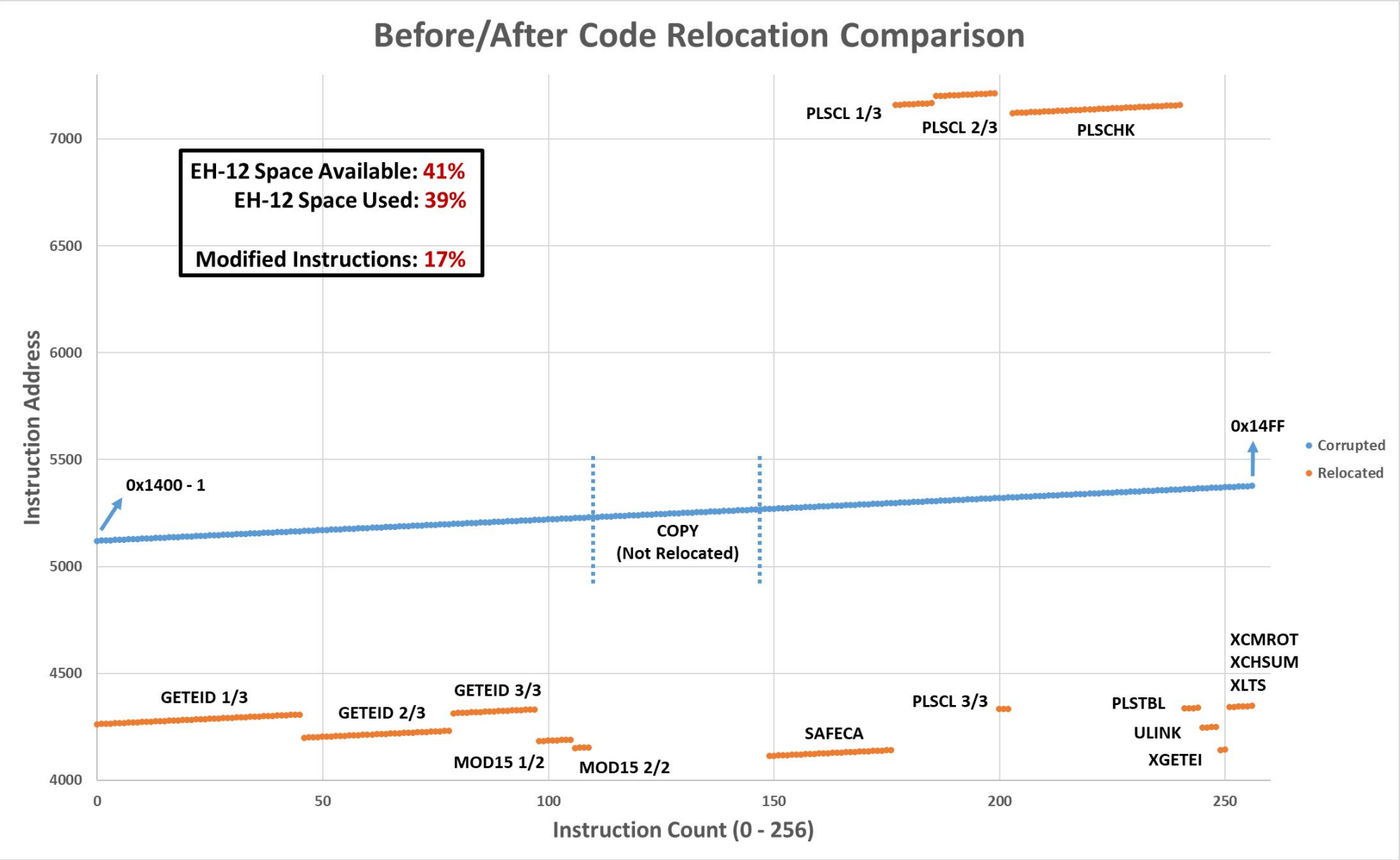
- Received telemetry from Voyager 1 for the first time in five months!



Phase 2: Science Telemetry Patch

- Relocate the remaining corrupted subroutines/data and the references to them
 - Repeat steps used for EL-40 patch
- Problem - Not enough available space remaining in EH-12 memory
 - Flight team pointed us to a 1990 Interoffice Memorandum that mentioned free memory
 - Does look free in the source code listing, but some conflicting information
 - Verified by comparing MROs before/after anomaly that the location values have not changed
 - Used upper address areas to avoid jumping between higher and lower memory
 - 1BD0 – 1BFF and 1C1F – 1C2D
 - Used a portion of higher memory that is not write protected. Lower risk compared to the complications of using lower memory
- May 10: Science Telemetry Patch uplinked to spacecraft
- May 19: First confirmation that this patch worked (CR-5T)
 - Remaining 2 data modes (GS-4B, PB-14) fully confirmed by July 27

Statistics (thanks to Armen Arslanian)



Statistics (thanks to Armen Arslanian)

